



كلية الهندسة المعلوماتية

السنة الثانية

تمارين إضافية Linked List

د. عملي مشترك



RB Informatics; 12/11/2024

خوارزميات

ملاحظة : جميع التوابع التالية يجب أن تكتب داخل class الـ Linked List

جميع الحلول هي حل طلابي

تمرين 1 : كتابة تابع لإضافة عنصر في نهاية الـ Linked List

```
public void insertLast (int data) {  
    Node newNode = new Node(data) ;  
    if (head == null) {  
        head = newNode ;  
        return ;  
    }  
    Node current = head ;  
    while (current.next != null) {  
        current = current.next ;  
    }  
    current.next = newNode ;  
}
```



تمرين 2 : كتابة تابع لإضافة عقدة إلى الـ linked list بعد عقدة معينة

```

public void insertAfterSpecificValueNode(int valueNode, int data) {
    Node newNode = new Node(data);
    if (head == null) {
        // the list is empty
        System.out.println("The list is empty. The new node will be inserted as the first element");
        head = newNode;
        return;
    }
    Node current = head;
    while (current != null) {
        if (current.data == valueNode) {
            // the value node founded
            newNode.next = current.next;
            current.next = newNode;
            return;
        }
        current = current.next;
    }

    // the value node isn't found
    System.out.println("Value not found. The new node will be inserted at the end of the list");
    current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = newNode;
}

```

data : هي العقدة الجديدة المراد إضافتها

valueNode : هي العقدة المراد إضافة العقدة الجديدة بعدها

```
public boolean insertPos(int pos, int data) {
    Node newNode = new Node(data);
    if (head == null) {
        // list is empty
        System.out.println("The list is empty.");
        return false;
    }
    if (pos == 1) {
        // node inserted at the beginning
        newNode.next = head;
        head = newNode;
        return true;
    }
    Node current = head;
    int count = 1;
    // Traverse to the node just before the desired position
    while (current != null && count < pos - 1) {
        current = current.next;
        count++;
    }
    // Position out of range
    if (current == null) {
        System.out.println("Position out of range.");
        return false;
    }
    // the position is in range so we insert the node in it
    newNode.next = current.next;
    current.next = newNode;
    return true;
}
```



تمرين 4 : كتابة تابع لحذف عنصر من موقع محدد (أي أننا نرسل موقع العنصر في الـ Linked List و ليس قيمته)

```
public boolean deletePos(int pos) {
    if (head == null) {
        // list is empty
        System.out.println("List is empty");
        return false;
    }
    if (pos == 1) {
        // delete the first node
        head = head.next;
        return true;
    }

    Node current = head;
    int count = 1;
    // Traverse to the node just before the desired position
    while (current != null && count < pos - 1) {
        current = current.next;
        count++;
    }

    //position is out of range
    if (current == null || current.next == null) {
        System.out.println("position is out of range ");
        return false;
    }
    current.next = current.next.next;
    return true;
}
```

تمرين 5 : كتابة تابع للبحث عن عنصر في الـ Linked list بطريقة عودية

شرح الكود :

عند كل عقدة current يتم فحص إذا كانت تحوي الـ data المطلوبة أم لا

: base case

إذا وجدت الـ data المطلوبة أو

إذا كان الـ current يشير الـ null ، عندها إما أنه تم الوصول إلى نهاية الـ linked list أو أن الـ linked list خالية

: recursive case

أن نتقل للـ node التالية و نكرر العملية

```
public boolean isExist(int value, Node
current) {
    if (current == null)
        return false;
    if (current.data == value)
        return true;
    return isExist(value, current.next);
}
```

ملاحظة :

عند استدعاء التابع يجب إرسال :

- الـ value و هي القيمة المراد البحث عنها
- الـ head ، و الذي سيكون قيمة الـ current في الاستدعاء الأول

مثال :

```
public class Main {
    public static void main(String[] args) {
        LinkedList l = new LinkedList();
        l.insertLast(1);
        l.insertLast(2);
        l.insertLast(3);

        System.out.println(l.isExist(3, ll3.head)); // --> true
    }
}
```

تمرين 6 : كتابة تابع يعيد حجم ال Linked list

```
public long length() {  
    Node temp = head;  
    int length = 0;  
    while (temp != null) {  
        length++;  
        temp = temp.next;  
    }  
    return length;  
}
```

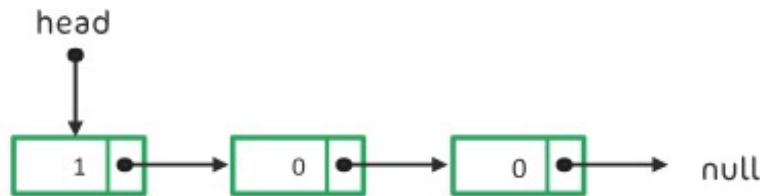


تمرين 7 : كتابة تابع يعيد العنصر الأوسط في ال linked list (و إذا كان هناك عنصرين يقعان في الوسط فيعيد العنصر الأوسط الثاني)

```
public int middleElement() {  
    if (head == null) {  
        return 0;  
    }  
  
    Node mid = head;  
    Node current = head;  
  
    while (current != null && current.next != null) {  
        mid = mid.next;  
        current = current.next.next;  
    }  
  
    return mid.data;  
}
```

تمرين 8 : بفرض أنه لدينا linked list قيمة كل عقدة فيها إما 0 أو 1. تمثل هذه الـ linked list عدد في النظام الثنائي، حيث أن الـ head يشير إلى الـ MSB في هذا العدد. اكتب تابع يعيد هذا العدد في النظام العشري.

مثال:



بالتالي $(100)_{\text{binary}} \leftarrow$

فالتابع يجب أن يعيد : $(4)_{\text{decimal}}$

■ طريقة 1 :

فكرة الحل أننا سنحسب عدد عناصر العدد في الـ binary ثم نحوله إلى decimal بالطريقة التالية :

مثال :

$$(100)_{\text{binary}} = 1 * 2^2 + 0 * 2^1 + 0 * 2^0 = (4)_{\text{decimal}}$$

```
public int binary2decimal() {
```

```
// your code here
```

```
if (head == null) {
```

```
    return 0;
```

```
}
```

```
int decimal = 0, i = 1;
```

```
Node cur = head;
```

```
while (cur.next != null) {
```

```
    cur = cur.next;
```

```
    i *= 2;
```

```
}
```

```
cur = head;
```

```
while (cur != null) {
```

```
    decimal += (cur.data * i);
```

```
    i /= 2;
```

```
    cur = cur.next;
```

```
}
```

```
return decimal;
```

```
}
```

شرح الكود :

في الحلقة الأولى قمنا بالوصول إلى آخر

عصر في الـ linked list و حساب الـ 2^n و

هو تثقيل الـ MSB أي أول عنصر في الـ

(Linked List

ثم أعدنا الـ current ليؤشر إلى أول عقدة

في الـ Linked list (أي الـ head)

في الحلقة الثانية حولنا العدد من binary

إلى decimal بدءاً من أول node

طريقة 2 :

فكرة الحل أنه سنمر على عناصر الـ Linked List بدأ من الـ MSB إلى الـ LSB (أي بدأ من الـ head حتى آخر عقدة) و نحول العدد من الـ binary إلى decimal باستخدام الطريقة التالية :

مثال : $(100)_{\text{binary}}, d = 0$ $d = d * 2 + 1 = 0 * 2 + 1 = 1$ $d = d * 2 + 0 = 1 * 2 + 0 = 2$ $d = d * 2 + 0 = 2 * 2 + 0 = 4$

```
public int binary2decimal(){
    Node current = head;
    int decimal = 0;
    while (current != null) {
        decimal = decimal * 2 + current.data;
        current = current.next;
    }
    return decimal;
}
```



تمرين 9 : بفرض أن الـ linked list المعطاة تمثل رقم معين، فيجب كتابة تابع يقوم بإضافة 1 إلى هذا الرقم.

مثال :

بفرض لدينا الرقم 1999 فسيتم تمثيله بالـ linked list :

 $1 \rightarrow 9 \rightarrow 9 \rightarrow 9$

فعند إضافة 1 إليه يجب أن تصبح الـ linked list كالتالي :

 $2 \rightarrow 0 \rightarrow 0 \rightarrow 0$

شرح الكود :

```

public void addOneToNum (){
    if (head == null) {
        // list is empty
        return;
    }

    Node LastToVisit = null;
    while (true) {
        Node curr = head;
        Node prev = null;
        while (curr.next != null && curr !=
LastToVisit) {
            prev = curr;
            curr = curr.next;
        }
        if (curr.data < 9) {
            curr.data++;
            break;
        }
        curr.data = 0;
        LastToVisit = prev;

        if (curr == head && curr.data == 0) {
            Node newNode = new Node(1);
            newNode.next = head;
            head = newNode;
            break;
        }
    }
}

```

- ننشئ 2 pointers أحدهم curr و يشير إلى ال head، و الآخر prev و يشير إلى null
- الحلقة الخارجية اللانهائية (تكسر الحلقة عند تحقق أحد الشروط داخلها) داخلها:
 - حلقة داخلية نمر فيها على عناصر ال linked list تكسر بأول دخول للحلقة الخارجية عندما يصبح ال curr يشير إلى آخر عقدة و ال prev يشير إلى العقدة ما قبل الأخيرة (أما إذا دخلنا إلى الحلقة الخارجية مرة أخرى فستكسر الحلقة الداخلية عندما curr = LastToVisit
 - إذا كانت العقدة التي يشير إليها curr أصغر من 9 ← نقوم بزيادة curr.data بمقدار 1 و ننهي الحلقة الخارجية .
 - إذا كان curr.data يساوي 9 ← نجعل curr.data = 0 و LastToVisit = prev و نكرر العملية السابقة (أي ندخل بالحلقة الخارجية مرة أخرى، لكن في هذا الدخول ستكسر الحلقة الداخلية عندما يصبح curr = LastToVisit ليس عند الوصول إلى آخر عقدة)
- إذا كان ال curr=head و curr.data = 0 أي عندها نكون وصلنا إلى أكبر خانة في الرقم و قيمتها 9، فعند جمع 1 لها سنجعل قيمتها 0 و نضيف node جديدة إلى بداية ال linked list قيمتها 1
(مثل : 9 → 9 → 9 تصبح : 0 → 0 → 0 → 1)

The End